

rec13: STL

▼ [Hide Assignment Information](#)

Instructions

rec13 - STL

Note: next week's recitation, rec14, is **check out in lab only**. You will check out and submit your code **in lab**. TAs will **not** be grading any submissions after the lab.

Topics

- Collections: vectors and lists
- Half-open range
- Iterators
- Auto
- Generic Algorithms
- Lambdas and Lambda capture
- Templated Functions
- Collections: maps and sets

Attachments

- pooh-nopunc.txt
- rec13_start.cpp
- output.txt

General notes

- Please note, do *not* use the keyword `auto`, except where specified.
- For every task in your code, please put a comment to identify which task number it is. This will help both you and the TA who looks at your code.
[I am providing a main that has done much of this for you, but it would be good to do the same for any additional code, e.g. functions or classes, that you need to write.]

Some Vector and List Basics

1. Create a vector with various values.
 - Print it out using the ranged for. (For later exercises, it may simplify your life if there are an **even number** of elements.)
2. Copy this into a `list`, using the list constructor that takes a half-open range, provided

by `begin()` and `end()`.

- **Print this list out.**

3. Sort the original vector, then print both it and the list out.

- `sort` also takes a half-open range. Here we will provide it the vector's `begin()` and `end()`
- Note that sorting the list cannot be done the same way. We will discuss why further down and how to work around that.

4. Write a loop to print every *other* element of the vector, starting with the first element.

The `range` for won't work for this, instead use an "index-based" for-loop.

5. *Attempt* to do the same with the linked list.

- It doesn't compile.
- Why? No `op[]` for list!
- Why didn't they provide `op[]` for list? Easy to write, right? (In fact you've done it.) They omitted it just so we wouldn't try to use it in a loop like this. (How did you write it? Each call to `op[]` required walking from the front of the list. To loop over N items this way would take approximately $N^2 / 2$ steps. Ouch!)

Iterators

6. Use iterators to redo task 4, i.e display every *other* element of the vector.

- Remember how to declare the *type* of an iterator.
- Note that the list iterators we are using here support: copy constructor, `!=`, `++`, `--`, and dereference.
- Iterators for vectors support `==`, `!=`, `++`, `--`, dereference, *and* adding on and arbitrary integer values, just as we could do with pointer arithmetic. An iterator of this type sort is known as a **random access iterator**. For this task, use the random access ability of vector's iterators.
- You will want the **vector's length to be even** for this task.

7. Repeat the previous task for the list.

- In the previous task, you bumped the iterator by 2, but that won't work with the list iterator. List iterators do not support random access. Same reason that lists don't support the index operator.
- So figure a work around in order to repeat the previous task for the list.
- **Again**, you will want the list's length to be even for this task.

8. As we have now seen that there is more than one kind of iterator.

- Lists have *bidirectional* iterators which support moving back and forth, but *only* by using the increment and decrement operators. Vectors have iterators that do all of that but also support an addition operator that takes as a second argument an integer, so that you can do the equivalent of pointer arithmetic with vector iterators. Again, these are known as random access iterators. (There are *other* types, too!)
- As you know, most sort algorithms require random access. Because of that, we cannot use the algorithm `sort` with the iterators for a list. Of course, lists can be sorted (hopefully you sorted lists in your data structures course) and the list class

has its own sort *method*. The method does not take any arguments, its purpose being to sort the entire list.

- Sort your list with the **list class's sort method**. And of course show that it worked.

9. Write a function that is passed in a list of ints and prints the list using iterators.

[Naturally, it should be passed a constant reference to the list.]

- First try using `list::iterator` as before and observe that you get a compilation error.
- Try again using `list::const_iterator`.
 - It works!
 - But what a pain!

10. Repeat the previous task of printing the list, but this time instead of using iterators, use a ranged for.

Happily, this works just as before. And no complication with `const`.

Auto

11. So, using ranged-for is great where we can get away with it. But when we can't, it can be annoying having to specify those messy types.

- In comes `auto`!!!
- Let's write a function that will print every other item in the list. Instead of worrying about how to specify the type for the iterator, just use the keyword `auto`. The code is easier to write and actually easier to read, while still being statically typed. Yes, it is statically typed, but the compiler is doing the work using *type inference*

12. Write a function to find an item in the list, returning an iterator for where it is.

- The function will take a list, again passed by `const` reference, and an `int` to search for.
- Do not use `auto`.
- What did you have to use for the return type?
- What should / did you return when the search failed?
- Be sure to test the function with an item that is there and an item that is not there.

13. Ok, modify the for loop to use `auto`. That works, great.

- What about the return type? That still has the ugly `const_iterator`. Try to use an `auto` there. If your compiler supports not just `c++11`, but also `c++14`, then it will work.

Generic Algorithms

Be sure to check if your call to any of the algorithms "succeeded"!

14. Use the algorithm `find` to do the same as the previous step.

- Note the need for the `include` (actually, we already included it to use `sort`).
- Remember that the generic `find` is *not* passed a collection, instead it is passed a "half-open range".

- Always remember to handle the possibility that what you are looking for is not found!

15. Use the algorithm `find_if` to locate the first even number in the vector and in the list and print them out.

- Note that you first have to define a *function* that takes an `int` and returns a `bool` indicating whether the integer was even or not.
- When you call `find_if`, you will pass in the usual half-open range defining where to search, followed by the function. How do you pass a function? Just pass its name. No, don't imagine that the *code* for the function is being passed in, the name just specifies what function we are calling.
- Of course, the vector / list that you are searching may not have any even numbers, so the code that checks the return value of `find_if` must be prepared for such a condition.
- Again, always remember to handle the possibility that what you are looking for is not found!

16. Let's use a **lambda expression** for the problem of **finding the first even number**.

- A lambda expression is an "unnamed function". It has a bit of an odd syntax. The simplest example would be:

```
[] { cout << "Hello Lambda!"; }
```
- You could actually run that. Just put it on a line in `main`. And put a pair of parentheses after it, because after all if that's a function, and we still have to *call* it. So `main` *could* look like:

```
int main() {
    [] { cout << "Hello Lambda!"; }();
}
```

- So, what have we learned? The syntax for defining a lambda expression requires a pair of square brackets. Inside the square brackets there may be specified a list of variables that would be "captured". That does not affect us right now, so I will not explain any more about it.
- But what we haven't seen that we do need is how to provide a parameter list. That's because the function we wrote didn't need one. And the point of lambda expressions is to minimize what you need to write. So if you don't need any parameters, then you didn't even need a parameter list! If you do need parameters, then the parameter list would go right after the square brackets. I'll show you the use of a lambda expression taking two ints and printing the results. Try putting the following line in your program. It should print 9.

```
[] (int a, int b) { cout << a + b << endl; } (4, 5);
```

- What if we want to return the sum instead of printing it?

```
int result = [] (int a, int b) { return a + b; } (4, 5);
```

```
cout << "the result is: " << result << endl;
```

- Note that something else seems to be missing? The return type was never stated! Again, C++ is trying hard to learn how to reduce how much you have to type. It is clear to the compiler that this lambda expression is returning an int so it doesn't require that you say so.
- If you did want to say what the return type for that lambda expression was supposed to be, then the above code would instead look like:

```
int result2 = [] (int a, int b) -> int { return a + b; } (4, 5);  
cout << "the result is: " << result2 << endl;
```

- Yes, the return type shows up just before the body, after the optional parameter list. With the arrow which has nothing to do with a pointer. Weird, but you don't usually need to mention the return type. Lambda expressions are usually short and clear.
- So, now that you know about lambdas, **write one to use in the call to find_if so you can locate the first even number in the collection,**

17. And now let's make our use of the lambda more general. For the test if something was even, we knew that the "divisor" was going to be two. What if we don't know? Let's ask the user what value to use? But how can the lambda expression know what value to use? Remember, in our call to find_if, the function we use can only be passed a single argument, i.e. the number you are testing.

- Lambda expressions have a really cool feature, called lambda capture.
- In the square brackets, you can put the name of a variable that you have already defined. Then you can use that variable in the body of the lambda expression.
- So ask the user for a value, then capture that value in your lambda by putting it in the square brackets, and finally use the value in the body of the lambda.
- I should mention that lambda capture can be by value, as we are doing above,

18. Define a dynamic array (i.e. on the heap), the same size as the list and the vector.

- Use the copy algorithm to fill it either from the list or the vector.
 - Copy is passed the half-open range that it is copying *from* and the location of the first item it is copying *to*.
- Again, print its contents.
- Use find on the array to do the same tests we did before.
What do we have to pass in to specify the half-open range?
- Don't forget to free the space from the heap!!!

Templated functions

19. Implement your own version of the **find algorithm** (call it ourFind) for lists of ints.

- Remember what the **algorithm** is passed.
- This will be a *function*, **not** a function *template*.
- Put a print statement in your function so that we know it is yours
- show that it works, using the same tests you used with the library version.

20. Rewrite our `Find` as a function template. You do not have to comment out the function you wrote in the previous task.

- In the template, change the print line to say you are in the template
- Test your template using the vector.
- Run your test again on the list. Notice that C++ used your function, not the template.

That's a good thing!

Collections

Note that the attached output file has been abbreviated for the output of tasks 21-23. Yes, your output will be much longer.

21. We will read a file, listing all of the tokens, but just once for each distinct token. A text file is attached to this recitation for this purpose.

- Solve this using a vector of strings to track what you have seen.
- For each token that you read from the file check if it is in the vector and if it isn't then add it to the vector.
 - To make your life easy, use the STL algorithm `find` to check if the token is in the vector.
- After filling the vector, display its size and contents. (But first do remember to close the file!)
- Running this on the attached file, you will likely notice your program hesitate a moment while it fills the vector.
- Incidentally, when I ran my solution, I found 2,278 distinct words.

22. Repeat the previous task but instead of using a vector to hold the tokens, use a set to do so.

- You will have to include the set header file, i.e. `#include`
- You define a set the same way as you define a vector, simply type set and then place the kind of things the set should hold in angle brackets.
- You *could* test if an item is in a set, using the set method `find`. If the item is not found, `find` will return the same value as set's `end()` method. (Note this is not the generic find algorithm. Why not?)
- However, to add an item to a set, just use set's method: `insert`. If the item is already in the set, then nothing will change, so you don't actually need to first test if it is there.
- If you want to know whether insert succeeded, i.e. added a new item to the set, you can look at the return value. The insert method returns a `pair`, a templated struct, where the field `first` is an iterator to the item inserted, and the field `second` is a `bool` which is true if something was added, and false otherwise.
- You will see that this version finishes *much* faster. *And* the elements will be displayed in sorted order without your having to write any code to handle the sorting! (How do you think that was done? What does it suggest might be the underlying implementation of set.)

- I hope you remembered to close the file as soon as you were done reading it!

23. Finally, we will repeat a problem from early in the semester where we tracked word positions from a document. Remember how we created a struct to hold the token and a vector of positions, where a position indicated what token *number* it is, not the character position (which would be more work for you). Then maintained a vector of these struct instances? This time we will use a map, much like a dictionary in Python, to map from the tokens to their vectors. The goal is to print out each word in the document, one per line, together with the token positions for that word.

- First we need to define our map variable. (Of course, we have to have included the map header file, just as we did with the set.)
 - The map requires *two* types, not just one. We have to say what we are mapping from, here a string, and what we are mapping to, here a vector of ints. So we could define a variable wordMap, as follows:
`map<string, vector<int>> wordMap;`
- We can access each element of the map, using the square bracket operator. `wordMap["fred"]` would return the value, a vector of ints, for fred's entry in the map.
- If the key "fred" hasn't been entered into the map yet, C++ automatically creates an entry with a value the is "default initialized". In the case of our map, since the value is a vector, it will be initialized to an empty vector.
- All we need to do is `push_back` onto that vector the new position. The line:
`wordMap["fred"].push_back(17)`
would add the position 17 to fred's vector. And if fred wasn't there before, it will now be there with a vector holding just the single value 17. Almost like magic!
- Finally we need to be able to loop over the map. Using the `ranged for`, it is very easy. Each item in the map is a `pair`. Again, a pair has two fields `first` and `second`. In a map, the field `first` for each entry is the "key" (the word) and the field `second` is the "value" (the vector of positions). To keep your life simple, use the keyword `auto` to specify the type for the `ranged-for`.
- For each item in the map, first print out the word and then print out all of the positions. It would look best to do this all on one line for each word.
- Again, notice how quickly this runs and that the words appear in sorted order.
- And it was a lot easier to write this time than when we did it in class!
- And one last time, I hope you remembered to close the file as soon as you were *done reading* it!

Turn in

A single file `rec13.cpp`